04 iDeliveryFood Prompts Backoffice Servicos

Regras desta revisão — aplicação integral

Este documento conserva o conteúdo e os módulos do PDF anexado, com alterações nas instruções afetadas, sem resumo. As regras abaixo se aplicam a todos os módulos e critérios de aceite deste arquivo.

Mobile nativo e testes por plataforma

Android: Kotlin e Jetpack Compose; iOS: Swift e SwiftUI, projetos separados. Sem app híbrido/React. Implementar e testar cada plataforma independentemente: unitários de domínio e ViewModel, integração API, UI, deep links, sessão, permissões, offline e acessibilidade. CI Android com toolchain apropriado; CI iOS em runner macOS com Xcode. Assinatura e contas de distribuição são externas ao Git. Falta de ambiente Apple deve aparecer como pendência, nunca como build/teste concluído. Web/PWA permanece entrega web separada, jamais substitui os apps nativos.

MCP exato fornecido pelo proprietário

{
"mcpServers": {
"supabase": {
"serverUrl": "https://mcp.supabase.com/mcp?project_ref=dwsoochnfxldfvcaijie&features=docs%2Caccount%2Cdatabase%2Cdebugging%2Cdevelopment%2Cfunctions%2Cbranching%2Cfeedback"
}
}
}

SQL remoto e seed

SUPABASE — PROJETO CORRIGIDO E EXECUÇÃO REMOTA: projeto autorizado dwsoochnfxldfvcaijie. Configure o MCP com o JSON exato desta revisão e autenticação segura. Inspecione o schema, crie/altere tabelas, índices, constraints, funções, RLS, grants, Storage e demais recursos necessários; execute os SQLs no Supabase por migrations versionadas e verifique o resultado real. Não entregue somente arquivos SQL sem aplicá-los. A senha anteriormente exposta deve ser rotacionada; nunca reproduzi-la ou versioná-la. Não gravar tokens no Git. Identifique o ambiente antes de inserir seed: usar desenvolvimento/teste. Em produção ou ambiente desconhecido, confirmar antes de gravar dados sintéticos. Não executar DROP/TRUNCATE/reset remoto ou alterações destrutivas sem aprovação específica. Registrar migration, resultado, validação e plano de recuperação sem segredos.

DADOS FICTÍCIOS PERSISTIDOS — SEED OBRIGATÓRIO: criar supabase/seed.sql com dados sintéticos coerentes para desenvolvimento/teste e executá-lo no Supabase autorizado. Tudo que representa dados de negócio deve persistir no Supabase: lojas, produtos, adicionais, preços, horários, pedidos, eventos e métricas derivadas. NUNCA colocar dados chumbados no HTML/páginas, componentes, classes dos apps, arrays locais ou respostas falsas de API. UI consulta banco/API com RLS; vazio real continua sendo estado válido. Seed idempotente com IDs estáveis, relações válidas e marcação de dados de teste; executá-lo duas vezes em teste sem duplicação. Não usar dados pessoais reais nem senhas no seed. Identidades de teste via Auth suportado, segredos fora do Git. Seed não dispara cobranças ou mensagens externas. Dados fictícios não entram em produção. Doubles/fixtures de unitários são permitidos só nos testes isolados, nunca importados pelo runtime. Traduções e configuração estática não são dados de negócio; cache apenas derivado do backend. Provedor ausente não pode retornar sucesso fictício.

GitHub depois dos testes

O Antigravity deve executar commit e push em https://github.com/JuanBMMontesinos/iDeliveryFood.git depois de testar o código e revisar diff e segredos. Verificar branch/remotes e preservar histórico; não

usar force-push, mudar visibilidade ou contornar proteção. Respeitar gates de aprovação do ambiente e autenticar por mecanismo seguro. Executar testes locais antes do push e acompanhar CI remoto depois; falha de CI requer correção e novo push, sem alegar release aprovado. Confirmar SHA remoto. Push não é deploy nem publicação nas lojas.

Idioma principal

INTERFACE E i18n: inglês (en) como idioma principal e padrão inicial; usuário pode mudar para português brasileiro (pt-BR), com preferência persistida no Supabase para conta autenticada. Android usa recursos strings/plurals, iOS usa String Catalog/localization, web usa solução i18n com chaves, parâmetros, pluralização e fallback en. Não concatenar frases traduzíveis; traduzir validações, erros públicos, acessibilidade e notificações. Formatar datas/números/moeda por locale; BRL e região de operação não mudam só porque a interface está em inglês. Responsividade e acessibilidade; estados loading/vazio/erro/offline/sem permissão. Testar troca de idioma, persistência, fallback e paridade de chaves. O termo técnico é i18n (não i8n).

Documentação e segurança preservadas

Manter documentação completa, testes unitários, integração, RLS/pgTAP, e2e e controles de cybersegurança dos módulos originais. Atualizar arquitetura, ERD, migrations aplicadas, seed, API, matriz de permissões, instruções nativas de build/teste, i18n, CI/CD, operação e evidências. Não incluir segredos; não declarar certificação de segurança sem validação independente.

Conteúdo integral revisado do documento anexado

iDeliveryFood — Backoffice e serviços

Como usar

PACOTE REVISADO — PROMPTS POR APLICATIVO, PORTAL E MÓDULO

Este material substitui o pacote anterior. É uma especificação e conjunto de comandos para o agente construir o iDeliveryFood incrementalmente; não é código já implementado nem certificação de produção.

Os prompts cobrem explicitamente app web e app mobile do cliente, app mobile do entregador, portal do parceiro, gestor de pedidos, portal de redes, console logístico, área de frota, administração central e backoffices financeiro, atendimento, comercial, marketing, relatórios, risco e privacidade, além da infraestrutura comum.

Como usar: cole as instruções permanentes antes do prompt do módulo. Em uma nova sessão, repita-as e peça ao agente para ler docs/DELIVERY_STATUS.md e os contratos existentes. Execute um módulo por tarefa, respeitando dependências. Não envie todos para execução simultânea. IDs indicam módulos, não obrigatoriamente a ordem numérica.

Projeto Supabase corrigido: dwsoochnfxldfvcaijie. Executar migrations e seed no ambiente de desenvolvimento autorizado conforme esta revisão. Rotacionar a senha exposta; arquivos não contêm senha nem URI credenciada.

Não há publicação GitHub, acesso ao banco ou implantação realizada por este pacote. Contas externas, contratação de serviços, credenciais seguras, revisão jurídica e aprovações permanecem responsabilidade do proprietário.

Instruções permanentes

INSTRUÇÕES PERMANENTES — COPIAR COMO CONTEXTO EM TODAS AS SESSÕES

Você é a equipe de engenharia do iDeliveryFood dentro do Antigravity. Execute uma etapa por vez e produza código funcional, não apenas planos, protótipos ou descrições. Inspecione o repositório e siga os contratos/ADRs existentes. Não substitua stack ou reescreva módulos prontos sem justificar e obter aprovação. Identifique esta entrega pelo ID do módulo, mantenha docs/DELIVERY_STATUS.md atualizado e não afirme conclusão sem evidências.

REPOSITÓRIO: alvo solicitado pelo proprietário:

https://github.com/JuanBMMontesinos/iDeliveryFood.git. Verifique primeiro acesso, conteúdo, branch, remotes e estado local. Não suponha que exista ou esteja vazio. Após testes aprovados, executar commit e push ao repositório solicitado e aplicar migrations no Supabase autorizado. Respeitar aprovações do ambiente; não alterar visibilidade, usar force-push ou executar operações destrutivas sem autorização específica. Antes de qualquer

publicação: revisar diff, testes e secret scan; não expor credenciais ou dados pessoais.

SUPABASE — PROJETO CORRIGIDO E EXECUÇÃO REMOTA: projeto autorizado dwsoochnfxldfvcaijie. Configure o MCP com o JSON exato desta revisão e autenticação segura. Inspecione o schema, crie/altere tabelas, índices, constraints, funções, RLS, grants, Storage e demais recursos necessários; execute os SQLs no Supabase por migrations versionadas e verifique o resultado real. Não entregue somente

arquivos SQL sem aplicá-los. A senha anteriormente exposta deve ser rotacionada; nunca reproduzi-la ou versioná-la. Não gravar tokens no Git. Identifique o ambiente antes de inserir seed: usar desenvolvimento/teste. Em produção ou ambiente desconhecido, confirmar antes de gravar dados sintéticos. Não executar DROP/TRUNCATE/reset remoto ou alterações destrutivas sem aprovação específica. Registrar migration, resultado, validação e plano de recuperação sem segredos.

DADOS FICTÍCIOS PERSISTIDOS — SEED OBRIGATÓRIO: criar supabase/seed.sql com dados sintéticos coerentes para desenvolvimento/teste e executá-lo no Supabase autorizado. Tudo que representa dados de negócio deve persistir no Supabase: lojas, produtos, adicionais, preços, horários, pedidos, eventos e métricas derivadas. NUNCA colocar dados chumbados no HTML/páginas, componentes, classes dos apps, arrays locais ou respostas falsas de API. UI consulta banco/API com RLS; vazio real continua sendo estado válido. Seed idempotente com IDs estáveis, relações válidas e marcação de dados de teste; executá-lo duas vezes em teste sem duplicação. Não usar dados pessoais reais nem senhas no seed. Identidades de teste via Auth suportado, segredos fora do Git. Seed não dispara cobranças ou mensagens externas. Dados fictícios não entram em produção. Doubles/fixtures de unitários são permitidos só nos testes isolados, nunca importados pelo runtime. Traduções e configuração estática não são dados de negócio; cache apenas derivado do backend. Provedor ausente não pode retornar sucesso fictício.

ARQUITETURA DE REFERÊNCIA: monorepo com backend/web TypeScript e workspace pnpm para apps/customer-web (Next.js), apps/partner-web (Next.js), apps/admin-web (Next.js), apps/api (NestJS) e apps/worker. Mobile com código NATIVO separado: apps/customer-android e apps/courier-android em Kotlin/Jetpack Compose; apps/customer-ios e apps/courier-ios em Swift/SwiftUI. Se algum novo app mobile for habilitado, também terá implementações Android e iOS independentes. NÃO usar React Native, Expo, Flutter, Ionic, Capacitor ou solução híbrida; não encapsular páginas web como app. Compartilhar contratos OpenAPI e especificações de domínio, não UI/runtime TypeScript com os apps. packages/contracts, domain, authz, api-client, ui-web, config são para backend/web; clientes nativos próprios por plataforma. Gradle e Xcode gerenciam seus projetos. Validar versões estáveis, lockfiles e ADR de migração se já houver código. Logística e portais de rede/frota mantêm áreas web autorizadas próprias; um backend comum para todas as interfaces.

CONTRATOS: API versionada /v1 com OpenAPI e cliente tipado; Supabase Auth para identidade; PostgreSQL/Supabase como fonte de verdade; AuthZ no servidor e RLS/grants no banco para as superfícies acessíveis; Realtime somente em canais autorizados. Negócio e escrita privilegiada no backend; consultas diretas do cliente apenas quando explicitamente aprovadas e protegidas por RLS. Service-role e segredos exclusivamente no servidor, jamais bundles/public env. Roles não podem vir de metadata editável pelo usuário. Multi-tenant por organização e unidade; evitar caches compartilhados entre tenants. Finanças, auditoria e documentos sensíveis em superfícies restritas.
SEGURANÇA: MFA obrigatório para staff privilegiado; cookies/tokens apropriados a cada plataforma, validação server-side, CORS allowlist, CSP e headers na web, rate limits, proteção a CSRF/XSS/SQLi/SSRF/IDOR, uploads seguros, rotação/revogação, log redigido sem segredos/PII indevida. Mobile usa armazenamento seguro de tokens e permissões mínimas. Não armazenar dados de cartão. Endereço/localização/documentos com exposição e retenção mínimas. Mudança bancária, repasses e reembolsos acima da alçada exigem controle adicional. Threat model e checklist OWASP ASVS versionados como metas verificáveis, sem alegar certificação. Revisão jurídica e pentest externo continuam pendências de lançamento.
PADRÕES DE DOMÍNIO: centavos inteiros para BRL; datas persistidas UTC e exibidas America/Sao_Paulo; validação de timezone de horário de loja; UUIDs/IDs não substituem autorização; transações, constraints, índices e idempotency keys para mutações críticas; outbox e consumidores

idempotentes para tarefas assíncronas. Máquina de estados e histórico de eventos de pedido separados dos estados de pagamento e entrega. Não chamar provedor externo dentro de transação longa. Estado final do PSP exige validação de assinatura, correlação e reconciliação. Não confiar em preço, comissão, papel ou status enviado pelo cliente.

INTERFACE E i18n: inglês (en) como idioma principal e padrão inicial; usuário pode mudar para português brasileiro (pt-BR), com preferência persistida no Supabase para conta autenticada. Android usa recursos strings/plurals, iOS usa String Catalog/localization, web usa solução i18n com chaves, parâmetros, pluralização e fallback en. Não concatenar frases traduzíveis; traduzir validações, erros públicos, acessibilidade e notificações. Formatar datas/números/moeda por locale; BRL e região de operação não mudam só porque a interface está em inglês. Responsividade e acessibilidade; estados loading/vazio/erro/offline/sem permissão. Testar troca de idioma, persistência, fallback e paridade de chaves. O termo técnico é i18n (não i8n).

QUALIDADE: cada módulo traz testes unitários de regras, integração local com Supabase, casos de acesso negado e multi-tenant, contratos e e2e aplicáveis. CI exige lint, typecheck, testes, RLS/pgTAP, build e secret scan. Propor meta inicial de cobertura de 80% em código de domínio e cobertura de decisões críticas, sem manipular métricas; não é resultado garantido. Testes ausentes ou pulados explicitamente relatados. Nunca executar testes com cobrança ou dados sintéticos em produção.

FINAL DE CADA ETAPA: listar arquivos alterados; migrations; endpoints; telas; permissões; comandos executados e resultados reais; cobertura; evidências; riscos; pendências de configuração; atualização de docs e próximos passos. Se faltar credencial/aprovação, continuar apenas no trabalho seguro local e registrar BLOQUEADO, não CONCLUÍDO. Após os testes e secret scan, realizar commit e push no GitHub alvo, respeitando gates do ambiente e confirmando SHA remoto.

M12 — Pagamentos, PSP, webhooks e estornos

PROMPT M12 — Pagamentos, PSP, webhooks e estornos

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M10; escolha e credenciais seguras do PSP.

OBJETIVO: Implementar cobrança real com limites claros por provedor.

IMPLEMENTAÇÃO E TELAS

Primeiro registrar ADR de PSP: Pix/cartão, split 1:1/1:N efetivamente contratado, onboarding/KYC, liquidação, taxas, sandbox, refund/chargeback, webhooks e conciliação. Não assumir split para entregador se serviço não oferece. Usar SDK/tokenização/checkout do PSP, sem PAN/CVV no servidor ou log. Integração runtime real; provider ausente impede cobrar. Sandbox identificado é permitido em ambiente de testes, não como pagamento de produção.

Assinar/verificar webhook e replay window conforme documentação do PSP; correlacionar merchant/pedido/valor/moeda; idempotência e eventos fora de ordem; não confiar no redirect do browser para marcar pago. Reconciliar por consulta ao PSP e backoffice. Proteger segredo, usar retries

com backoff e dead-letter. Refund parcial/total sujeito a alçada, idempotência e ledger, com estados pendentes/falhos. Evitar saldo virtual que alegue custódia não contratada.

DADOS, CONTRATOS E ACESSOS

payment_attempts, provider_events, refunds; referências/metadata mínimas e sem dados de cartão.

TESTES OBRIGATÓRIOS

Unitários com doubles só em testes; integração sandbox real quando credenciais disponíveis; assinatura inválida, replay, timeout pós-cobrança, duplicate webhook, chargeback, refund repetido e valor divergente.

CRITÉRIOS DE ACEITE

Pagamentos e estornos com estado/evidência verificável; pendências sem credencial marcadas BLOQUEADO, não sucesso.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M12.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

M13 — Backoffice financeiro e conciliação

PROMPT M13 — Backoffice financeiro e conciliação

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M10, M12; M14 para entregas.

OBJETIVO: Construir área financeira própria em admin-web e leituras autorizadas nos portais.

IMPLEMENTAÇÃO E TELAS

Telas: transações, ledger, comissões, fretes, subsídios/cupons por financiador, recebíveis por estabelecimento, ganhos de entregador, repasses previstos/executados, conciliação PSP, divergências, refund/chargeback e exportação. Regras financeiras versionadas por contrato e vigência. Ledger de partidas dobradas com lançamentos balanceados, imutáveis e reversões auditadas; valores em centavos. Não calcular receita como total bruto sem distinguir repasses. Payout somente por provedor/contrato confirmado e autorização com segregação de função; nenhum botão pode simular transferência. Fechamentos e relatórios explicam corte de data, moeda, status e origem. Não substituir contabilidade oficial nem emitir documento fiscal sem solução aprovada.

DADOS, CONTRATOS E ACESSOS

ledger accounts/journals/entries, settlements, payout references, reconciliation runs/matches; acesso financeiro restrito.

TESTES OBRIGATÓRIOS

Balanceamento, reversão, replay, diferença de centavos, conciliação parcial, comissão de vigências distintas, permissão e aprovação de payout; leitura operador negada.

CRITÉRIOS DE ACEITE

Todo total tem trilha até transação real; divergências visíveis; nenhuma transferência sem confirmação do PSP.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M13.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

M14 — Logística, despacho e console operacional

PROMPT M14 — Logística, despacho e console operacional

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M10, M03; modelo de entrega aprovado.

OBJETIVO: Implementar backend logístico e console no admin-web.

IMPLEMENTAÇÃO E TELAS

Telas: pedidos aguardando coleta, entregadores disponíveis reais, mapa somente com integração e coordenadas reais, fila sem entregador, atribuir/reofertar/reatribuir com motivo, atrasos, ocorrências e histórico. Diferenciar entrega própria da loja e entrega plataforma; sem rastreio, não alegar posição real. Calcular rota/cotação/ETA via provedor escolhido ou regra simples explicitamente aprovada; nunca inventar ETA. Despacho manual primeiro; automático após política aprovada de elegibilidade/distância/capacidade/expiração. Uma entrega não pode ter duas atribuições ativas indevidas. Roteirização múltipla depende de política de prazo/capacidade, não habilitar por padrão. Geolocalização com retenção curta configurada, janela operacional e controles antiacesso indevido; incerteza de GPS tratada, não tomada como prova única.

DADOS, CONTRATOS E ACESSOS

delivery_tasks, offers, assignments, location samples, delivery events; eventos internos e app M11.

TESTES OBRIGATÓRIOS

Atribuição concorrente, entregador offline, expiração, reatribuição após coleta bloqueada ou controlada, coordenada antiga, recálculo e acesso restrito.

CRITÉRIOS DE ACEITE

Despacho real auditado; atrasos/sem cobertura visíveis; sem mapa ou motorista fictício.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M14.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

M16 — Administração central, acessos e cadastros

PROMPT M16 — Administração central, acessos e cadastros

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M03, M04, M10.

OBJETIVO: Construir a estrutura do backoffice em apps/admin-web.

IMPLEMENTAÇÃO E TELAS

Dashboard por papel com métricas reais e definição dos indicadores. Módulos de usuários, organizações, lojas, entregadores, documentos/aprovações, contratos/condições, bloqueios/suspensões motivados, filas de operação, pedidos e histórico. RBAC com escopo regional/unidade quando aprovado e negativa por padrão. Não usar superadmin como conta diária. Contas individuais, convites, MFA, expiração/revogação e auditoria de ações sensíveis. Proibir impersonation silenciosa; se implementada após ADR, consentimento/regra, TTL, banner e log explícitos. Busca por dados pessoais limitada e rastreável; exportação com escopo/alçada. Auditoria não editável por operador comum.

DADOS, CONTRATOS E ACESSOS

/admin/users, merchants, couriers, approvals, access, audit; finanças e logística continuam módulos próprios.

TESTES OBRIGATÓRIOS

Matriz de papéis, tentativa de grant próprio, alteração de dado bancário, ação sem justificativa, exportação sem permissão e busca cross-tenant não autorizada.

CRITÉRIOS DE ACEITE

Cada menu corresponde a endpoint autorizado; papéis separados para operação, suporte, financeiro, comercial, marketing, risco, auditoria e técnico.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M16.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

M17 — Atendimento, suporte e resolução de incidentes

PROMPT M17 — Atendimento, suporte e resolução de incidentes

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M10, M16; M12/M13 para reembolso.

OBJETIVO: Construir console de atendimento com escopo e alçadas.

IMPLEMENTAÇÃO E TELAS

Telas de ticket, fila/prioridade, contexto do pedido, timeline, participantes, mensagens, anexos privados, categorização, responsável, SLA configurado, escalonamento e conclusão. Entrada de suporte no cliente, parceiro e entregador; entregador em rota priorizado conforme regra aprovada. Canais externos só quando configurados. Atendente vê dados necessários e pode solicitar ação; refund/payout segue fluxo financeiro e alçada, não edição direta de estado. Registro de autoria, motivo e evidências; anonimização/retenção conforme política. Não enviar mensagens reais a destinatário sem ação autorizada e confirmação adequada na operação.

DADOS, CONTRATOS E ACESSOS

support_cases/messages/attachments, action_requests e auditoria; acesso por participante/atribuição e staff autorizado.

TESTES OBRIGATÓRIOS

Acesso a ticket alheio, arquivo malicioso, reembolso acima da alçada, ação repetida, SLA e mascaramento de PII.

CRITÉRIOS DE ACEITE

Chamados reais integrados aos apps e ações rastreáveis; histórico preservado sem exposição indevida.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M17.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

M18 — Comercial, contratos e onboarding de parceiros

PROMPT M18 — Comercial, contratos e onboarding de parceiros

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M16, M07.

OBJETIVO: Separar prospecção comercial do financeiro e da operação.

IMPLEMENTAÇÃO E TELAS

Área comercial: leads cadastrados legitimamente, pipeline, responsáveis, contato com base/finalidade documentada, proposta, condições e vigência, aprovação de comissão/desconto excepcional e conversão para organização/loja após aceite. Contrato não é válido só por clicar em botão: integração de assinatura ou aceite com evidências precisa de aprovação jurídica. Comercial não pode executar repasse nem acessar documento além do necessário. Não carregar lista comprada/fictícia nem realizar disparos externos automaticamente.

DADOS, CONTRATOS E ACESSOS

leads/proposals/contracts/rule versions/approvals, relação com organização sem duplicar cadastro.

TESTES OBRIGATÓRIOS

Condição sem aprovação, vigência futura, acesso financeiro negado e conversão duplicada.

CRITÉRIOS DE ACEITE

Condições auditadas e versionadas; contrato/template pendente de jurídico claramente sinalizado.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M18.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais,

limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

M19 — Marketing, cupons, avaliações e campanhas

PROMPT M19 — Marketing, cupons, avaliações e campanhas

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M04, M10, M13, M16.

OBJETIVO: Implementar promoções com impacto financeiro explícito.

IMPLEMENTAÇÃO E TELAS

Criar campanhas e cupons com elegibilidade, validade/fuso, limite por cliente/pedido/orçamento, valor mínimo, teto e financiador (plataforma/loja) aprovados. Aplicação atômica para evitar ultrapassar orçamento por concorrência. Relatório por resgate real, não alcance inventado. Avaliação somente de pedido elegível e autor autenticado, uma avaliação por política, moderação rastreável e resposta do parceiro; nada de estrelas iniciais falsas. Segmentação minimizada/consentimento conforme base jurídica; comunicação externa só via fornecedor configurado. Publicidade patrocinada e assinatura apenas após contratos/preço aprovados, com identificação transparente, não implementar cobrança fictícia.

DADOS, CONTRATOS E ACESSOS

promotions, redemptions/reservations, reviews/moderation, campaigns e ledger de subsídio.

TESTES OBRIGATÓRIOS

Resgate simultâneo, cupom expirado, orçamento, cancelamento/reversão, elegibilidade, autoavaliação indevida e permissão de marketing.

CRITÉRIOS DE ACEITE

Desconto refletido na cotação, pedido e conciliação; métricas reais e comunicações configuradas.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M19.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

M20 — Notificações, tempo real e integrações externas

PROMPT M20 — Notificações, tempo real e integrações externas

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M03, M10; eventos dos demais módulos.

OBJETIVO: Centralizar comunicação confiável e rastreável.

IMPLEMENTAÇÃO E TELAS

Implemente outbox transacional/worker, templates versionados por evento, preferências e consentimento aplicável; canais in-app e push, e-mail/SMS/outros somente com provedor aprovado. Realtime não substitui armazenamento durável: reconectar e recuperar eventos por cursor. Filas com idempotência, backoff, limites, dead-letter e replay controlado. Revogar device tokens, diferenciar iOS/Android/web, não expor tokens ao parceiro. Conteúdo mínimo na tela bloqueada. ERP/PDV/iFood: conectores somente se contratados e homologados; não são dependência do marketplace próprio. Webhooks externos com assinaturas, tenant e logs redigidos. Não ativar loop de mensagens em teste ou disparo para clientes reais.

DADOS, CONTRATOS E ACESSOS

outbox, notification preferences/deliveries/devices, connector configs em secret store e eventos correlacionados.

TESTES OBRIGATÓRIOS

Duplicatas, falha/retry, token revogado, canal sem credencial, tenant leakage, webhook inválido e replay sem reenvio indevido.

CRITÉRIOS DE ACEITE

Entrega registrada de forma honesta: enfileirado/enviado/confirmado/falhou conforme evidência do provedor, sem inventar confirmação.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M20.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

M21 — Relatórios, indicadores, risco e privacidade

PROMPT M21 — Relatórios, indicadores, risco e privacidade

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M13, M16, M17, M19.

OBJETIVO: Criar relatórios confiáveis e rotinas de governança.

IMPLEMENTAÇÃO E TELAS

Relatórios: volume de pedidos, GMV versus receita líquida claramente definidos, conversão somente com eventos reais e consentimento aplicável, taxa de cancelamento com denominador, prazo de preparo/entrega com timestamps, disponibilidade, SLA e margens conforme custos registrados. Dashboard sem dados mostra vazio. Exportações auditadas com expiração e acesso restrito; agregação não pode reidentificar indevidamente. Risco: regras explicáveis e revisão humana de bloqueio, sem alegar IA antifraude inexistente.

Privacidade: inventário de dados por finalidade/base a validar com jurídico, acesso/portabilidade/correção/exclusão, retenção por categoria, bloqueio de exclusão quando retenção legal aplicável, anonimização, incidentes e contatos. Auditor/encarregado vê somente o necessário; consentimento não usado como base genérica para tudo.

DADOS, CONTRATOS E ACESSOS

queries/materialized views autorizadas, analytics events minimizados, privacy requests, retention jobs e risk decisions.

TESTES OBRIGATÓRIOS

Fórmulas com conjuntos de teste isolados, fuso/corte de data, dados faltantes, exportação negada, retenção e anonimização sem destruir trilha financeira necessária.

CRITÉRIOS DE ACEITE

Cada KPI tem fórmula/fonte/período; relatório reproduzível; políticas jurídicas pendentes explicitadas.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M21.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.